

Explain point 3 more thoroughly. How does the amount of data points affect the nearest neighbor summaries? Isn't the amount of data points propagated to the calculations?

Regarding point 4, the whole calculation looks like this:

```
for col in eachcol( training_summaries )
    temp = loss( col, mean_summary, inv_cov_mat )
    mean_standardization += temp
    sd_standardization += (temp -
mean_standardization)^2
end
```

Show more ▾

Worked for 2m 36s >

Yes. There are a couple of places where I would refine what I said earlier, especially for the ID nearest-neighbour ratio. I went back through the relevant functions in your uploaded toolbox. EmpiricalLikelihoodToolbox

Point 3: why does changing the number of points matter for nearest-neighbour summaries?

Your intuition is partly correct: **the number of data points is propagated into the distance calculation.** But that does not make the resulting statistic independent of the number of points. Quite the opposite: nearest-neighbour distances depend on how many candidate neighbours exist.

Your normal training procedure with `StandardResampling` does

```
julia
ntot_half = div(ntot, 2)

x_inds = index_cache[1:ntot_half]
y_inds = index_cache[(ntot_half+1):end]
```

so if $N = 1000$, roughly speaking you compare

500 points against 500 points.

But the `n_summaries <= 1` branch in `calculate_simulated_statistics` does

```
julia
summaries(
    view_in,
    index_cache,
    index_cache,
    R0_all,
    Rsim_container,
    buffers
)
```

so the MCMC calculation compares

1000 observed points against 1000 simulated points.

The fact that 1000 is propagated into `pairwise!` / `knn` is precisely why this isn't the same statistic.

Raw nearest-neighbour distances

Suppose the simulated points have some approximately constant local density $f(x)$ in d dimensions, and a point x has m possible neighbours.

For a small radius r ,

$$P(Y \in B(x, r)) \approx f(x)V_d r^d,$$

where V_d is the volume of a unit d -dimensional ball.

The probability that **none** of the m points lie within that distance is approximately

$$P(R_1 > r) \approx (1 - f(x)V_d r^d)^m \approx e^{-mf(x)V_d r^d}.$$

So the typical first-neighbour distance behaves approximately as

$$R_1 \propto m^{-1/d}.$$

Likewise,

$$R_k \sim \left(\frac{k}{mf(x)V_d} \right)^{1/d}.$$

Therefore doubling the number of candidate neighbours from $N/2$ to N changes the typical distance approximately by

$$\frac{R_k(N)}{R_k(N/2)} \approx 2^{-1/d}.$$

For example:

dimension	factor after doubling number of points
$d = 1$	0.50
$d = 2$	0.707
$d = 3$	0.794
$d = 10$	0.933

So for something like your **Chamfer distance**, which is based directly on nearest-neighbour distances, this matters substantially.

If target training says

$$C_{\text{target}} = C(X_{500}, Y_{500})$$

and MCMC says

$$C_{\text{sim}} = C(X_{1000}, Y_{1000}),$$

they are not statistically comparable without accounting for the sample-size effect.

But ID is more subtle

This is where I would qualify my original statement.

Your ID statistic is not the raw distance. You calculate

$$\frac{R_{k+1}}{R_k}.$$

Under the local homogeneous Poisson approximation,

$$R_k \propto m^{-1/d},$$

and therefore

$$\frac{R_{k+1}}{R_k} \approx \frac{m^{-1/d} A_{k+1}}{m^{-1/d} A_k} = \frac{A_{k+1}}{A_k}.$$

The direct dependence on m cancels.

So **for ID specifically, your intuition is substantially correct**: the ratio is deliberately much less sensitive to point density/sample size than a raw nearest-neighbour distance.

In the ideal asymptotic setting, its distribution can essentially be sample-size invariant.

It is not perfectly invariant for finite samples because boundaries, varying density, finite m , large k/m , and geometry all matter. But I overstated the sample-size problem for ID in my previous answer.

So what is actually wrong with your `n_summaries == 1` branch?

There are three separate issues.

First, for Chamfer and other raw nearest-neighbour statistics, it genuinely changes the statistic.

Second, for ECDF/CIL statistics it changes mainly the **sampling variance**, even if the expected statistic remains approximately the same. A 500-point ECDF is noisier than a 1000-point ECDF. A pair-distance ECDF likewise has different finite-sample variability.

Third—and this is the decisive software issue—it makes `n_summaries` change the definition of the calculation:

```
n_summaries = 1
    full data

n_summaries = 2
    two random half-data calculations

n_summaries = 10
    ten random half-data calculations
```

Normally one would expect `n_summaries` to control Monte Carlo precision only, not switch between two different estimators.

And for CIL/ID there is also your fixed-size buffer problem. The buffer was allocated assuming the half-half split, yet the shortcut sends full-size index arrays into it.

My recommended fix

I would simply remove the special case:

```
julia
```